

Memoria Proyecto de Programación Orientada Al Rendimiento

Grupo 81 Equipo 01
Álvaro Guerrero Espinosa - 100472294
Adrian Cortázar - 100475860
José Antonio Verde Jiménez - 100472221
Tomás Mendizábal - 100461170

Índice

1 Diseño . . . 2
1.1 `sim` . . . 2
1.1.1 `ArgumentParser` . . . 2
1.1.2 `FileIO` . . . 2
1.1.3 `Constants` . . . 2
1.1.4 `IO` . . . 2
1.1.5 `Vector3` . . . 2
1.1.6 `Particle` . . . 3
1.1.7 `Block` . . . 3
1.1.8 `Grid` . . . 3
1.2 `fluid` . . . 4
1.3 `utest` . . . 4
1.4 `ftest` . . . 5
2 Optimización . . . 6
2.1 Optimizaciones de `Grid` . . . 6
2.2 Optimizaciones de `Block` . . . 6
2.3 Optimizaciones generales . . . 7
3 Pruebas realizadas . . . 8
3.1 `ArgumentParser` . . . 8
3.2 `FileIO` . . . 8
3.3 `IO` . . . 8
3.4 `Vector3` . . . 8
3.5 `Particle` . . . 9
3.6 `Block` . . . 9
3.7 `Grid` . . . 9
3.8 Pruebas funcionales . . . 9
4 Evaluación de rendimiento y energía . . . 10
5 Datos de participación . . . 12
6 Conclusión . . . 12

1 Diseño

En esta sección se discutirán las decisiones tomadas más importantes para el diseño de cada uno de los componentes de este proyecto.

1.1 `sim`

Este componente contiene la biblioteca de la simulación, la cual está dividida en 8 módulos, cada uno encargado de una tarea.

1.1.1 `ArgumentParser`

Este módulo se encarga de leer e interpretar los argumentos pasados al programa, realizando su validación. En caso de haber algún problema, termina la ejecución del programa. En caso de no haber ningún problema, devuelve los valores leídos. Se decidió que terminase la ejecución del programa en caso de error ya que esto facilita la realización de las pruebas unitarias relacionadas con estos casos.

1.1.2 `FileIO`

Este módulo se encarga de gestionar los archivos de entrada y salida. Inicialmente comprueba que sus rutas sean válidas y se puedan abrir en el respectivo modo, lee el archivo de entrada, y valida que su cabecera sea válida. Tras esto, permite cargar el contenido de este archivo en la malla de la simulación, validando que los datos leídos sean correctos. Además, al finalizar la simulación, también se encarga de escribir el resultado en el archivo de salida. Si alguna de las validaciones falla, termina la ejecución del programa, lo cual se decidió porque facilita la realización de las pruebas unitarias relacionadas con estos casos.

1.1.3 `Constants`

Este módulo únicamente contiene la definición de las constantes usadas en la simulación que se conocen en tiempo de compilación.

1.1.4 `IO`

Este módulo contiene utilidades para leer y escribir archivos, y mostrar mensajes por pantalla de una manera más simple. Estas utilidades usan templetización variádica para permitir indicar en la misma llamada todos los datos a leer/escribir, con una interfaz similar al print de python.

En este módulo se necesitan desactivar un aviso de `clang-tidy` en 2 líneas del código: `pro-type-reinterpret-cast`. Esto es necesario porque las funciones de leer y escribir archivos usan secuencias de bytes, por lo que es necesario usar `reinterpret_cast()` para convertir los bytes leídos a los tipos de datos que codifican, y para convertir los datos a escribir a bytes.

1.1.5 `Vector3`

Este módulo contiene la definición de un vector de 3 dimensiones de un tipo genérico. Además, también define los operadores de este tipo de objetos usados por el programa (suma/resta, multiplicación/división por un escalar, división por otro vector coordenada a coordenada, normal al cuadrado, limitación de los valores a unos rangos y comparación de igualdad). Este se decidió crear ya que hay muchas operaciones en el programa que usan operaciones vectoriales.

1.1.6 `Particle`

Este módulo contiene la definición de las partículas del sistema, almacenando la información variable de cada una de ellas, así como su identificador.

1.1.7 `Block`

Este módulo contiene la definición de los bloques de la malla de la simulación. Cada uno de estos contiene un vector con las partículas asignadas a él, así como los métodos para añadir partículas, y obtener el número de partículas almacenadas y la información de una partícula. Inicialmente se eligió usar una implementación de array de estructuras por simplicidad.

1.1.8 `Grid`

Este módulo contiene la definición de la malla de la simulación y constantes desconocidas en tiempo de compilación, así como los métodos necesarios para añadir partículas (añadiéndolas al bloque correspondiente según su posición), leer todas las partículas en orden según su identificador, y ejecutar un paso de tiempo de la simulación. Se decidió implementar estos pasos aquí ya que modifican el estado interno de la malla de la simulación.

La malla contiene un vector de bloques que se puede indexar con un `Vector3` de las coordenadas de un bloque. De esta forma, se puede implementar una matriz de 3 dimensiones con un vector unidimensional, evitando indirecciones innecesarias y estableciendo de manera clara el orden de almacenamiento de los elementos. Además, también se incluye un método para indexar estos vectores según las coordenadas de una posición en la simulación, calculando primero las coordenadas del bloque y usando luego la anterior funcionalidad.

En cuanto a la ejecución de un paso de tiempo de la simulación, este se divide en 8 pasos:

1) Iniciación de densidades y aceleraciones: en este paso se establece el valor inicial de estos atributos para todas las partículas.
2) Incremento de densidades: en este paso se itera por todos los pares de partículas en bloques adyacentes para calcular su densidad.
3) Transformación de densidad: en este paso se transforma la densidad de cada partícula sumando una constante y multiplicando el resultado por otra.
4) Transferencia de la aceleración: en este paso se itera por todos los pares de partículas en bloques adyacentes para calcular su aceleración.
5) Interacciones con las paredes: en este paso se itera por las partículas en los bloques de los bordes y se actualiza su aceleración.
6) Movimiento de las partículas: en este paso se itera por todas las partículas y se actualiza la posición, velocidad, y gradiente de la velocidad de cada una.
7) Colisiones con con las paredes: en este paso se itera por todas las partículas en los bloques de los bordes, y se actualiza la posición, velocidad, y gradiente de la velocidad de las partículas fuera del recinto de la simulación.
8) Reposicionamiento de las partículas: en este paso se crea un nuevo vector de bloques, se reasignan las partículas a los bloques de este vector según su posición, y se reemplaza el vector de bloques de la malla con este. Se usa un nuevo vector para hacer esto ya que si esto se hiciese sobre el vector original podría haber problemas al reordenar los datos de una estructura de datos a la vez que se recorre.

Para realizar estas operaciones, se han generalizado los procesos repetidos en métodos auxiliares:

- Iteración por los bloques de la malla: este método itera por las coordenadas de cada bloque de la malla, y aplica una función pasada como parámetro a cada una.

- Iteración por bloques adyacentes a uno dado: este método itera por las coordenadas de cada bloque adyacente a uno dado (excluyendo el bloque dado), comprobando que sean válidas y aplicando una función pasada como parámetro a cada una de ellas. Inicialmente se decidió calcular los bloques adyacentes en cada ejecución en vez de almacenar los resultados al inicio por simplicidad, pero haciendo pruebas también nos dimos cuenta de que almacenar el resultado era más lento, Teniendo esto en cuenta, decidimos mantener la implementación original.

  Además, desde el inicio se decidió que, por simplicidad y rendimiento, esto se haría de tal forma que tras iterar por todos los bloques y seleccionar sus adyacentes, se hubiesen seleccionado todos los pares de bloques adyacentes exactamente una vez (optimización conocida como frente de onda). De esta forma se evita tener que comprobar si un par se ha seleccionado ya o no en cada iteración, ya que este caso no se llega a dar nunca.

  Para conseguir esto, se tienen que cumplir una propiedad sobre el conjunto de bloques adyacentes usado. Si el conjunto de desplazamientos (x, y, z) respecto del bloque dado a considerar como adyacentes para cualquier bloque es S, entonces:

  $$(x, y, z) \in S \Leftrightarrow (-x, -y, -z) \notin S$$

  Esta condición es necesaria y suficiente: $\forall A, B$ bloques adyacentes, si $\text{offset}(A, B) = (x, y, z)$ es el desplazamiento de A respecto de B, entonces $\text{offset}(B, A) = -\text{offset}(A, B) = (-x, -y, -z)$. Por lo tanto: $\text{offset}(A, B) \in S \Leftrightarrow (x, y, z) \in S \Leftrightarrow (-x, -y, -z) \notin S \Leftrightarrow \text{offset}(B, A) \notin S$, es decir, $\text{offset}(A, B) \in S \Leftrightarrow \text{offset}(B, A) \notin S$. Como resultado, el par de bloques adyacentes siempre se evaluará en exactamente uno de los dos casos. Esto también implica que S tendrá exactamente la mitad de los desplazamientos a bloques adyacentes.

- Iteración por pares de partículas en bloques adyacentes: este método utiliza los 2 métodos anteriores para iterar por todos los pares de partículas en bloques adyacentes sin repeticiones, y aplicarles una función pasada como parámetro. Para hacer esto, primero usa el primer método para iterar por todos los bloques. Tras esto, usa el segundo para iterar por todos los bloques adyacentes a este, y para cada uno aplica la función dada a cada par de una partícula del bloque original y otra del adyacente. Finalmente, itera por todos los pares de partículas sin repeticiones del bloque original y les aplica la función dada.

- Iteración por las partículas de un borde: método templetizado que itera por todas las partículas de un borde seleccionado con parámetros de template y aplica una función a cada una. Esto se hace para evitar definir la misma operación para cada posible borde. Los bordes se seleccionan con parámetros de template para generar funciones específicas para cada borde en tiempo de compilación, eliminando bifurcaciones innecesarias.

- Aplicación de una función a las partículas de cada borde: este método ejecuta el método anterior con una función pasada como parámetro para cada uno de los 6 bordes del recinto.

1.2 `fluid`

Este componente contiene la función `main` del programa principal. Simplemente se encarga de pasar los argumentos a `ArgumentParser`, inicializa la malla de la simulación, carga el archivo de entrada en esta usando `FileIO`, y ejecuta el número de pasos de tiempo pedido. Finalmente, usa `FileIO` para escribir el resultado en el archivo de salida.

1.3 `utest`

Este componente contiene todas las pruebas unitarias de la biblioteca de la simulación, con un archivo `.cpp` por componente a probar. Se escribieron pruebas para todos los módulos excepto para el de

constantes, ya que este módulo no contiene código ejecutable. Para realizar dichas pruebas se usa el framework de pruebas GoogleTest. Estas pruebas se describen más adelante en la sección de pruebas realizadas. Todas las pruebas se pueden ejecutar con el programa compilado `./utest/utest` sin argumentos.

1.4 `ftest`

Este componente contiene las pruebas funcionales de la simulación, implementadas como un script de `bash` que ejecuta el programa final con diferentes entradas y comprueba las salidas. Estas pruebas se describen en la sección de pruebas realizadas. Todas estas pruebas se pueden ejecutar con el script `./ftest/ftest.sh`, usando como primer y único argumento la ruta al ejecutable del programa (`fluid`)

2 Optimización

El tiempo de ejecución del diseño original es de una media de 14.876s para realizar 1000 pasos de tiempo con el modelo grande en avignon, calculada usando 20 repeticiones. Esta será la métrica a usar para evaluar los resultados de las optimizaciones. Tras analizar el problema, nos dimos cuenta de que las operaciones más costosas son aquellas que requieren iterar por pares de partículas (incremento de densidades y transferencia de la aceleración), por lo que las principales optimizaciones estarán en este área.

2.1 Optimizaciones de `Grid`

La primera optimización que aplicamos fue con respecto a la reserva de memoria. Para evitar reservar memoria para un nuevo vector de bloques en el reposicionamiento de partículas, decidimos almacenar 2 vectores de bloques: `front` y `back`. El vector `front` se corresponde con el vector original, mientras que el vector `back` es en el que se añaden las partículas durante el reposicionamiento. Tras finalizar esto, estos vectores se intercambian, permitiendo reusar el vector de bloques de la anterior iteración como vector `back` en la siguiente. Para poder hacer esto, además, tuvimos que añadir un método a los bloques que permitiese vaciarlos sin liberar su memoria reservada, y hacer esto antes de añadir las partículas en el reposicionamiento. Con esta optimización, el tiempo de ejecución bajó a 13.972s respecto del diseño original.

Tras esto, decidimos fusionar pasos de la simulación que podían hacerse juntos sin modificar el resultado (optimización de fusión de bucles). Primero nos dimos cuenta de que la inicialización de la densidad y la aceleración se podía realizar durante la carga inicial de las partículas y en el reposicionamiento, ya que estos datos se cargaban en caché en estos pasos de todas formas, y esto evitaba tener que recorrer todas las partículas otra vez. Tras esto, nos dimos cuenta de que como el paso de la transformación de las densidades solo requería sumarle a la densidad una constante y multiplicar el resultado por otra, este paso se podría eliminar inicializando la densidad a la constante de suma, y aplicando la propiedad distributiva a lo largo de los incrementos de la densidad del paso anterior. Con estas optimizaciones, el tiempo de ejecución bajó a 13.927s.

2.2 Optimizaciones de `Block`

La primera optimización fue con respecto a la reserva de memoria para el vector. Para evitar las múltiples copias que haría el vector a medida que se añaden partículas y se incrementa el espacio reservado, decidimos añadir un parámetro en el constructor que permitiese indicar el número de partículas para las que reservar memoria inicialmente. De esta forma, se puede usar una estimación del número de partículas de cada bloque para evitar las copias iniciales, aunque con un pequeño coste de memoria si se sobrestima la cantidad real. En nuestro caso, decidimos usar como estimación el número medio de partículas por bloque asumiendo una distribución uniforme, ya que es una estimación fácil y rápida de calcular. Con esto, el rendimiento mejoró a 13.872s.

Tras analizar el problema, nos dimos cuenta de que durante los pasos más lentos (iteración por pares de partículas en bloques adyacentes) no se usaban todos los atributos de las partículas. Más específicamente, en el paso de incremento de densidades sólo se necesitaba la posición y densidad de las partículas, y en el paso de transferencia de la aceleración se necesitaban la posición, densidad, velocidad, y aceleración. Además, el identificador de las partículas solo se necesitaba durante el reposicionamiento en bloques (para mover la partícula a otro bloque). Para evitar cargar datos innecesarios en la caché, decidimos cambiar la estructura interna de los bloques. Primero probamos a usar una estructura de arrays, mejorando el rendimiento a 13.494s.

Tras esto, probamos alternativas híbridas que juntasen atributos comúnmente usados juntos en un único vector, lo cual reduciría el número de vías necesarias para obtener un buen resultado y el tamaño

de los bosques en el vector de la malla (optimización de fusión de arrays). Primero juntamos la posición y densidad en un único vector, dando un mejor rendimiento de 12.0861s. Tras esto, decidimos combinar también la velocidad y aceleración, obteniendo en este caso un rendimiento de 11.596s. En este caso, en ambos de los procesos críticos solo se cargan los datos necesarios, y reducir el número de vectores accedidos más aún requeriría cargar datos innecesarios en el primero de los procesos críticos, por lo que consideramos esta implementación como óptima.

2.3 Optimizaciones generales

La penúltima optimización fue mover la definición de las funciones no templetizadas de grid a los archivos `.hpp`. Esto permite al compilador generar su código en línea, permitiendo realizar algunas optimizaciones adicionales. Aunque en general esto no tiene por qué mejorar el rendimiento, gracias a qué estas funciones solo se ejecutaban desde un punto del programa, sí que lo mejoró en nuestro caso, obteniendo un rendimiento de 11.258s. Por último, activamos la flag `-Ofast` del compilador, lo cuál mejoró el rendimiento a 10.9462s. Aunque esta flag no garantiza que el resultado de operaciones en coma flotante será correcto, como nuestro código no depende de las comprobaciones que desactiva, sí es válido en nuestro caso.

Con todo esto, consideramos que el rendimiento es óptimo ya que el tiempo total de ejecución está en aproximadamente 11s para el caso de 1000 pasos de tiempo, lo cual es un poco más de 4 veces más rápido que el tiempo de referencia dado de 45s para esta misma prueba.

3 Pruebas realizadas

3.1 `ArgumentParser`

Para este componente se han probado todos los flujos de ejecución: argumentos válidos, más y menos argumentos de los requeridos, parámetro de pasos de tiempo no numérico, y parámetro de pasos de tiempo negativo o 0. Para los casos de error, se ha usado un "Death Test" para validar que el programa termina con el código y mensaje de error correctos. Además, se ha probado que tras leer argumentos válidos, los métodos getter para leer los argumentos funcionan correctamente.

3.2 `FileIO`

Para este componente se han probado todos los flujos de ejecución de la validación de las entradas y lectura del archivo de entrada, y la escritura del archivo de salida. En todos los casos de error se ha usado un "Death Test" para validar la finalización del programa con el mensaje de error correcto.

Con respecto a la validación de las entradas, los casos posibles son los siguientes: archivo de entrada no se puede abrir para lectura, archivo de salida no se puede abrir para escritura, y el número de partículas en la cabecera del archivo de entrada es negativo, nulo o positivo. Además, se ha probado que los métodos getter devuelven los parámetros correctos leídos de la cabecera del archivo de entrada.

Con respecto a la lectura del archivo de entrada, los casos posibles son los siguientes: número de partículas en la cabecera mayor o menor que el número de partículas en el archivo, y número de partículas en la cabecera igual al número de partículas en el archivo. Además, también se ha probado que tras la carga del archivo se escribe un mensaje por la salida estándar con los parámetros de la simulación y el formato correcto, y que los datos cargados del archivo son los correctos. Con respecto a la escritura del archivo de salida, se ha comprobado que los datos escritos son los correctos comprobando que, si se escribe el archivo de entrada sin modificar sus datos, los 2 archivos coinciden byte a byte (gracias a que ambos archivos usan el mismo formato).

3.3 `IO`

Para este componente, para cada una de las funciones definidas, se ha probado que su comportamiento es correcto para los tipos de datos usados en el programa (`string`, `int`, `float`, `double`, y `Vector3`). Además, como son funciones variádicas, también se ha probado que funcionan correctamente al recibir distintas cantidades de parámetros, incluso cuando no todos los parámetros tienen el mismo tipo. En el caso de la función de lectura de un archivo, también se ha probado que si sobran datos en el archivo de un tamaño menor al del tipo que se está intentando leer, no se lee nada y se indica que se ha llegado al final del archivo.

Para realizar estas pruebas se ha tenido que desactivar el aviso `pro-type-reinterpret-cast` de `clang-tidy` en 2 líneas de las pruebas. El motivo de esto es el mismo que en la implementación de este módulo, ya que las pruebas necesitan leer/escribir archivos de forma independiente de `IO` al ser el código de este lo que se quiere probar.

3.4 `Vector3`

Para este módulo, se probaron todas las operaciones definidas. Ya que este tipo de datos está definido de forma genérica, cada prueba se realizó con cada uno de los tipos de datos usados por nuestro programa: `size_t`, `float` y `doubles`. Además, en el caso de la multiplicación y división por un escalar, como este también es genérico, la prueba se repitió también para cada tipo compatible con el usado por el vector (`int` para vectores de `int`, `int` y `float` para vectores de `float`, y `int`, `float` y `double` para vectores

de `double`). En todos los casos, se realiza la operación con vectores predefinidos y se comprueba que el resultado es el correcto.

3.5 `Particle`

Para este módulo se comprobó que los constructores inicializasen la partícula con los valores correctos, y que el operador de igualdad e inserción en `std::ostream` (`<<`) para imprimir la partícula funcionasen correctamente.

3.6 `Block`

Para este módulo, se probó que todos los métodos definidos funcionasen correctamente: añadir partículas, vaciar el bloque, getter de la cantidad de partículas almacenadas y cada uno de los atributos de una partícula, operador de indexación para leer una partícula, getter de todas las partículas, y operador de inserción en `std::ostream` (`<<`) para imprimir un bloque.

3.7 `Grid`

Para este módulo, se probaron los métodos públicos: inicialización de la malla, la inserción de partículas, los métodos getter, el método setter de los bloques (usado únicamente para facilitar otras pruebas), las funciones de indexación del vector de bloques, y los pasos de la simulación. Para la inicialización, se comprobó que se realizase correctamente, y que los métodos getter devolviesen los valores correctos. Para la inserción de partículas, se comprobó que se añaden al bloque correcto según su posición. Además, esto se realizó para partículas tanto dentro del recinto como fuera por cada uno de los bordes. Para los métodos getter, se comprobó que devolviesen los valores correctos, y para el método setter de los bloques que se actualizase correctamente el estado de la malla. Para las funciones de indexación, se probó que devolviesen el índice correcto, y en el caso de la indexación por posiciones de la simulación se repitió con los mismos casos que para añadir partículas.

Por último, para la realización de los pasos de la simulación, ejecutaron para 5 pasos de tiempo con los modelos `small` y `large`, y se comparó el estado de la simulación después de cada paso con el estado del archivo de trazas de referencia correspondiente. En caso de detectarse alguna discrepancia, el test fallaría. Como esta comparación era algo extensa, se decidió separar estas pruebas en 2 archivos: `traceio.hpp`, el cual contiene los métodos para cargar un archivo de trazas en memoria, y `simulation_test.cpp`, el cual contiene los tests junto con las funciones creadas para comparar los estados.

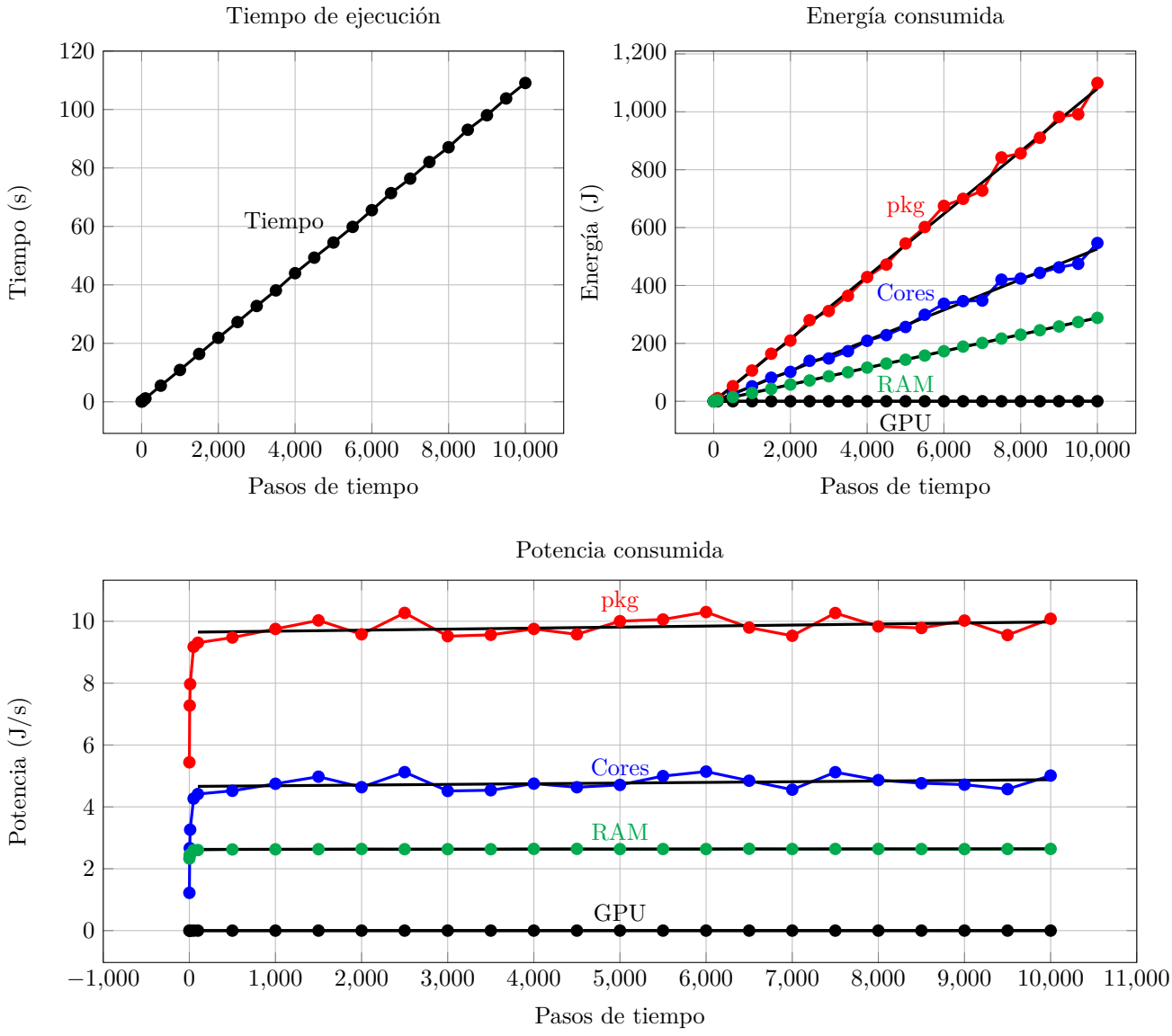
Cabe destacar que, como nuestra implementación del algoritmo difiere un poco de la descrita en el enunciado, aunque matemáticamente los resultados deberían ser idénticos, la precisión finita de los números en coma flotante hace que el resultado difiera ligeramente del contenido en las trazas. Para evitar este problema, necesitamos usar una tolerancia relativa de 10^{-8} en todas las comparaciones de `double`, la cual garantiza que el resultado de la salida en `float` es el correcto ya que estos tienen aproximadamente 7 decimales de precisión.

3.8 Pruebas funcionales

Para realizar estas pruebas, se creó un script de `bash` que ejecutase nuestro programa y comprobase que la salida es la correcta. Se incluyo como pruebas todos los casos de error ya descritos en las pruebas de `ArgumentParser` y `FileIO`, que para una entrada válida el mensaje escrito en la salida estándar con la información de la simulación es el correcto, y que la salida del programa con los modelos `small` y `large` para los primeros 5 pasos de tiempo coincide con los archivos de salida de referencia dados.

4 Evaluación de rendimiento y energía

Para realizar las mediciones, se ha ejecutado el programa con el modelo grande dado en un nodo del cluster de avignon, y se ha repetido la prueba múltiples veces para obtener la media. Para los casos con 2500 pasos de tiempo o menos, se han usado 20 repeticiones. De 2500 a 5000, se han usado 10 repeticiones. Por último, a partir de 5000 se han usado 5 repeticiones. Esto es debido al límite de 10 minutos de ejecución por trabajo en el cluster de avignon. Además, se ha calculado también una regresión lineal de cada uno de los datos para observar la tendencia, representada por las líneas negras en cada gráfica.



Como se puede observar, el tiempo y energía requerido crece aproximadamente linealmente con el número de pasos de tiempo ejecutados. Esto es debido a que la cantidad de trabajo por paso de tiempo es aproximadamente constante a lo largo de la simulación. El consumo de energía de GPU es siempre aproximadamente 0 ya que el programa no la utiliza.

En cuanto a la potencia, se puede observar que inicialmente cambia mucho, pero tras unos pocos pasos de tiempo se estabiliza, con algunas oscilaciones respecto de la media. Esto es debido a que cuando el número de pasos de tiempo es bajo, una parte significativa del tiempo de ejecución es la lectura del archivo de entrada y escritura del archivo de salida, y durante este tiempo la CPU tiene muy poca carga. Sin embargo, al aumentar la cantidad de pasos de tiempo, este tiempo pasa a ser despreciable en comparación con el tiempo requerido para realizar la simulación, durante el cual la CPU tiene mucha más carga. La potencia se estabiliza porque la carga durante la ejecución de la simulación es aproximadamente constante. Esto también explica por qué el incremento es mucho mayor para los núcleos y pkg que para la RAM, ya que la RAM sí que está en uso durante la lectura del archivo de entrada (se escriben los datos leídos en ella) y la escritura del archivo de salida (se leen los datos de ella). Como estos casos iniciales no son representativos de la tendencia de la potencia a medida que aumenta la cantidad de pasos de tiempo, no se han tenido en cuenta los datos obtenidos con menos de 100 pasos de tiempo para calcular la regresión lineal de la potencia.

5 Datos de participación

Participante	Tareas	Tiempo Dedicado
Adrián Cortázar Lería	Implementación de las clases `Particle`	1h
	Implementación de la clase `Vector3`	3h
	Pruebas Unitarias `FileIO`	4h
	Pruebas Unitarias `ArgumentParser`	2h
	Pruebas Unitarias `Grid`	2h
Álvaro Guerrero Espinosa	Implementación de la clase `Block`	1h
	Implementación de la iteración por pares de partículas en bloques adyacentes	1h
	Implementación de las métodos de indexación de la clase `Grid` e iteración por todos los bloques de la malla	2h
	Implementación de los pasos "incremento de la densidad" y "transferencia de la aceleración"	1h
	Implementación de los pasos "inicialización de la densidad y aceleración", "transformación de la densidad", y "movimiento de partículas"	1h
	Implementación del paso "reposicionamiento de las partículas"	1h
	Optimizaciones de la clase `Block`	4h
	Optimizaciones finales	1h
	Pruebas Unitarias `IO` y `Particle`	1h
José Antonio Verde Jiménez	Implementación de las clases `ArgumentParser`, módulo `Constants`, y función `main`	2h
	Implementación de la clase `IO`	1h
	Implementación de la estructura interna de la clase `Grid`, métodos para añadir partículas y getters	2h
	Implementación de la iteración por partículas en los bordes de la malla	2h
	Implementación de los pasos "interacción con las paredes" y "colisiones con las paredes"	1h
	Pruebas funcionales	2h
	Pruebas unitarias de los pasos de la simulación	2h
	Optimizaciones de la clase `Grid`	1h
Tomás Mendizábal	Implementación de la clase `FileIO`	4h
	Pruebas unitarias `Vector3`	3h
	Pruebas unitarias `Block`	2h

6 Conclusión

Tras la realización de esta práctica, nos ha resultado bastante entretenida debido a la grande carga de aprendizaje sobre el lenguaje de programación C++. Otro aspecto importante a resaltar, es el contexto de realización de la práctica, el cual nos ha parecido muy realista y útil. De la siguiente manera, se han adquirido las siguientes competencias:

- Mejora de eficiencia en el programa: Hemos aprendido a programar lo más eficiente posible en cuanto a mejoras del rendimiento. Por ejemplo, hemos podido mejorar la habilidad de encontrar defectos en el código orientados al rendimiento (utilizar los parámetros como referencias constantemente, valorar diferentes técnicas de implementación y escoger la que menos utiliza recursos, metaprogramación, optimizaciones relacionadas con el uso de la memoria caché, etc.).
- Aprendizaje de programación en C++: Aunque contamos con una base sólida de C, programar en C++ ha requerido su tiempo de aprendizaje debido a las características nuevas que incorpora y otras maneras de abordar ciertos conceptos. Ej: aprender a utilizar `std::unique_ptr` o `std::shared_ptr` en vez de los punteros de C o aprender los nuevos métodos de paso por

referencia en C++ versus el paso de punteros en C.
- Aprendizaje de trabajo en grupo: Con esta práctica, hemos aprendido a organizar las tareas dentro de un equipo lo más eficazmente posible y a mejorar las relaciones y tratos entre miembros.